

Data Engineer Technical Assessment

Contents

Data Engineer Technical Assessment	1
Business Scenario	1
Assignment Objective	2
Provided Materials	2
Requirements	2
1. Dimensional Modeling (Kimball)	2
2. ETL Pipeline (SSIS or Python)	2
3. Logging & Error Handling.....	3
4. Large Data Best Practices.....	3
5. Parameterization & Modularity.....	4
6. Source Control & Documentation	4
Submission Instructions	4

Business Scenario

The organization receives and processes extracted EMS records from various backend systems that are mapped to the National EMS Information System (NEMSIS) standard. These raw records conform to a common NEMSIS-compliant schema (aligned to current NEMSIS data dictionary and validation rules) and must be staged, validated, and transformed into a warehouse that supports BI dashboards and statistical reporting. The ETL should apply NEMSIS-specific strategies such as version-aware field mapping, codeset normalization to NEMSIS value sets, schema/validation-rule enforcement with reject quarantines, and crosswalk handling for local-to-NEMSIS element conversions, while preserving raw submissions for auditability. Analysts need consistent, historical, and well-documented facts and dimensions derived from this standardized NEMSIS foundation.

Assignment Objective

Design and implement an end-to-end ETL solution that ingests the provided sample Emergency Medical Services (EMS) dataset (CSV), stages it, cleans/transforms it, and loads it into a dimensional data warehouse using Kimball methodologies. Your solution should be production-minded: scalable, parameterized, modular, and robust to errors and data quality issues.

Provided Materials

- EMS sample dataset (CSV) attached in the application.
- You may assume a SQL Server target (preferred), but other relational targets are acceptable if justified.

Requirements

1. Dimensional Modeling (Kimball)

Create a dimensional model appropriate for EMS operational data.

Expectations

- Identify the business process and define the grain of the primary fact table(s).
- Create conformed dimensions where applicable.
- Include surrogate keys and proper handling of unknown/default members.
- Address slowly changing dimensions (SCD) as appropriate (Type 1/2 with rationale).

Deliverables

- ERD or star schema diagram (image or PDF).
- DDL scripts for warehouse objects (facts, dimensions, supporting tables).

2. ETL Pipeline (SSIS or Python)

Build an ETL that:

A. Stages

- Loads raw CSV into a staging schema/table(s).
- Preserves raw values for traceability.

B. Cleans/Transforms

- Applies data quality rules (null handling, type conversions, normalization, domain validation).

- Creates derived columns needed by your model (e.g., time keys, standardized codes, rollups).
- Produces reject/error outputs where records fail validation.

C. Loads DW

- Loads dimensions first, then facts.
- Uses set-based/bulk patterns suitable for large data volumes.
- Ensures referential integrity between facts and dimensions.

3. Logging & Error Handling

Include a logging strategy that captures:

- Run start/end timestamps
- Row counts (extracted, staged, inserted/updated, rejected)
- Error details (record id / row number, error type, message)
- ETL step status (success/failure)

Expectations

- Fail gracefully, with meaningful diagnostics.
- Ensure re-runability (idempotent or safe incremental loads).

4. Large Data Best Practices

Even though the sample is small, design as if it were large.

Consider:

- Batch loading and commit sizing
- Indexing strategy for staging vs DW
- Minimal row-by-row operations
- Partitioning or incremental load patterns
- Avoiding high-memory operations in Python
- SSIS fast-load, merge/upsert patterns

5. Parameterization & Modularity

Your ETL should be configurable without code changes.

Examples:

- File path / folder
- Target connection strings
- Batch size / load window
- Full vs incremental load
- Environment (dev/test/prod)

Expectations

- Modular design (clear separation of extract, stage, transform, load).
- Easy to extend with new EMS files/fields.

6. Source Control & Documentation

You must commit all work to a source control repo (GitHub, Azure DevOps, etc.).

Include:

- SQL scripts (DDL/DML)
- ETL code/package(s)
- Any config files / parameter files
- A clear README describing:
 - architecture and flow ○ how to run
 - end-to-end ○ assumptions and decisions ○
 - model grain and dimension/fact definitions ○
 - data quality rules ○ incremental strategy
 - (even if mocked) ○ logging/error handling
 - approach

Submission Instructions

1. Provide a link to your source control repository.

2. Ensure the repo contains all scripts, ETL code, and README.
3. Include your schema diagram in the repo.
4. If any setup is required, document it clearly.